



Aula 3

Redis

Chave-Valor, Arquitetura e Prática Guiada

Arquitetura in-memory, tipos de dados
e prática hands-on com Redis Cloud via Google Colab.

29 de agosto de 2026



O que vamos ver nesta aula



Retomada

Conexão com a Aula 2: NoSQL e categorias



Fundamentos chave-valor

Definição, arquitetura interna e anatomia de uma chave



Redis por dentro

Estruturas de dados, persistência e replicação



Ecossistema Redis

Redis Core x Redis Stack — módulos e expansões



Operações básicas

Comandos fundamentais de leitura e escrita



Prática guiada

Configurando e operando Redis Cloud via Google Colab



Atividade prática

Exercício hands-on de CRUD com Redis no Colab



Próximos passos

Gancho para a Aula 4

O que vimos na Aula 2

“De todas as categorias NoSQL, qual é a mais simples – e por isso a mais rápida?”

- 1 Limitações do modelo relacional e o Teorema CAP (CP x AP)
- 2 As quatro categorias NoSQL: chave-valor, documentos, colunas e grafos
- 3 Critérios de escolha entre categorias e o conceito de polyglot persistence

O banco chave-valor é o mais simples e rápido de todos.

O que é um banco de dados chave-valor?



Definição

Modelo de dados mais simples do universo NoSQL — cada registro é um par único (**chave** → **valor**).

A chave funciona como identificador direto; o valor é tratado como um bloco opaco (string, número, JSON serializado, blob).

- 1 Sem esquema**
Não há tabelas, colunas ou tipos predefinidos.
- 2 Sem linguagem de consulta complexa**
Apenas operações simples: GET, SET, DELETE por chave.
- 3 Extremamente rápido**
Leitura/escrita praticamente **O(1)** — não interpreta a estrutura do valor.
- 4 Consulta apenas pela chave**
Buscar por conteúdo exige varrer todos os dados ou manter índices externos.

Armazenamento in-memory: por que é tão rápido



Disco (tradicional)

Leitura/escrita passam por operações mecânicas ou de controlador — latência de milissegundos.



→ **Redis mantém os dados diretamente na RAM, ignorando os gargalos de leitura/escrita em disco — resposta em microssegundos.**

Latência sub-milissegundo

Ideal para caches, sessões e contadores em tempo real.

Throughput altíssimo

Milhares de operações por segundo em hardware modesto.

Trade-off: volatilidade

Dados em RAM são perdidos se o processo cair — por isso existem mecanismos de persistência (próximo slide).

Velocidade



Consultas em disco levam tempo. O usuário final acaba esperando.



Consultas em memória (RAM) aceleram drasticamente o tempo de resposta da API.

Arquitetura interna



Tabela hash

A chave é convertida em endereço de memória, permitindo acesso direto ($O(1)$) ao valor.



Single-thread + event loop

Um único processo executa os comandos em sequência, evitando condições de corrida.



Particionamento (sharding)

Em cluster, chaves são distribuídas entre nós por hash consistente.



Replicação primário-réplica

Réplicas mantêm cópias dos dados, oferecendo leitura distribuída e tolerância a falhas.



Persistência em disco

Snapshots (RDB) ou logs de escrita (AOF) garantem que os dados sobrevivam a um reinício.



Expiração (TTL)

Cada chave pode ter um tempo de vida; ao expirar, é removida automaticamente.

Uso da memória

A DESVANTAGEM

Dados armazenados na RAM são voláteis por natureza.



O RISCO

Em caso de falha do servidor, os dados em memória são perdidos.



O RESULTADO HISTÓRICO

Considerado arriscado demais como banco primário — dados críticos ficavam em disco.

COMO O REDIS MITIGA ESSE RISCO HOJE



RDB (snapshot)

Salva uma "foto" completa do dataset em intervalos configuráveis.



AOF (append-only file)

Registra cada operação de escrita em log, permitindo reconstrução total.



Replicação

Cópias em outros nós reduzem o impacto da perda de um único servidor.

A anatomia de uma chave (key naming)

Nome do recurso

Representa a coleção ou tipo de dado (ex.: um produto do catálogo).

Delimitador

A convenção padrão do Redis para separar escopos de forma legível.

ID único

O identificador específico daquele registro exato.

produto:101

As chaves podem ter qualquer nome, mas seguir o padrão **recurso:id** mantém seu banco organizado e altamente escalável (ex.: produto:102, produto:103).

O ecossistema: Redis Core vs. Redis Stack

Redis Core (o fundamento)



- ✓ Velocidade pura e extrema
- ✓ Estruturas de dados fundamentais (Strings, Sets, Hashes)
- ✓ Operações in-memory tradicionais

O foco principal de aplicações que precisam de cache absoluto.

Redis Stack (a expansão)

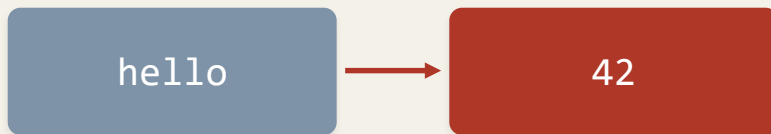


- ✓ Módulos e extensões adicionais
- ✓ Suporte nativo ao formato JSON
- ✓ Mecanismo de busca avançada (search) e consultas complexas

Transforma o Redis em um banco de dados multimodelos completo.

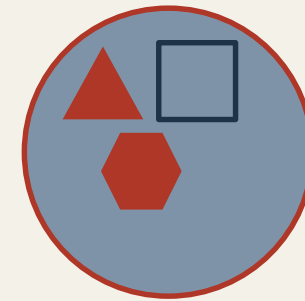
Tipos de dados I

Strings



O tipo mais básico. Usado para armazenar textos simples ou até mesmo números (que podem ser incrementados matematicamente).

Sets



Coleções desordenadas de Strings. A regra de ouro matemática: todos os valores inseridos dentro de um Set devem ser estritamente únicos.

Tipos de dados II: estruturas complexas



Title: Dune

Author: Herbert

Hashes: semelhantes a objetos na programação. São coleções de pares chave-valor perfeitas para representar entidades individuais e estruturadas.



Lists: coleções ordenadas de Strings. Diferente dos Sets, não exigem que os valores sejam únicos, permitindo duplicações.



Sorted Sets: semelhante aos Sets, mas cada valor possui um 'Score' (pontuação) associado, mantendo os itens ordenados automaticamente.

** Tipos avançados como Streams e Índices Geoespaciais também existem, mas estão fora do escopo inicial. **

Casos de uso principais do chave-valor



Cache

Ex.: cache de página

Reduz carga do banco principal guardando resultados de consultas frequentes com TTL curto.



Sessão de usuário

Ex.: e-commerce

Armazena o estado de login/carrinho entre requisições, com expiração automática.



Contadores

Ex.: contador de views

Incrementa valores atomicamente — visualizações, curtidas, rate limiting de API.



Filas simples

Ex.: fila de e-mails

Listas usadas como filas (FIFO) para processamento assíncrono de tarefas.

Exercício Prático

1

Ter o banco Redis Cloud pronto

Conta criada, banco gratuito ativo e credenciais (host, porta, senha) anotadas — ver Parte 1 a 3 do passo a passo.

2

Criar um notebook no Colab

Acesse colab.research.google.com e clique em File → New notebook.

3

Instalar o cliente Python do Redis

Na primeira célula, execute `!pip install redis` — a instalação ocorre na nuvem do Colab, não na sua máquina.

4

Conectar e testar

Crie a conexão com host/porta/senha e confirme com `r.ping()` — se retornar True, está tudo pronto.

Conectando e gravando dados via Python



CÉLULA DE CONEXÃO (executar uma vez no início do notebook)

```
import redis
```

```
r = redis.Redis(host="SEU_HOST", port=SUA_PORTA, password="SUA_SENHA", ssl=True, decode_responses=True)
print(r.ping()) # True confirma que a conexão funcionou
```

CREATE / UPDATE

```
r.set("produto:101", "Teclado Mecânico")
```

Equivalente a: SET produto:101 "Teclado Mecânico" — retorna True

READ

```
r.get("produto:101")
```

Equivalente a: GET produto:101 — retorna 'Teclado Mecânico'

DELETE

```
r.delete("produto:101")
```

Equivalente a: DEL produto:101 — retorna 1 (chave removida)

EXPIRAÇÃO

```
r.expire("produto:101", 60)
r.ttl("produto:101")
```

Equivalente a: EXPIRE + TTL — define e consulta o tempo de vida

Contadores e hashes via Python

Continuação da célula de conexão da tela anterior — o objeto `r` já está pronto para uso.

VERIFICAR EXISTÊNCIA

```
r.exists("produto:101")
```

Equivalente a: EXISTS produto:101 — retorna 1 (existe) ou 0

CONTADOR (INCR)

```
r.set("visitas:pagina", 0)  
r.incr("visitas:pagina")
```

Equivalente a: SET + INCR — incrementa atomicamente

HASH — CRIAR (HSET)

```
r.hset("usuario:1", mapping={  
    "nome": "Ana", "email": "ana@x.com"})
```

Equivalente a: HSET usuario:1 nome "Ana" email "ana@x.com"

HASH — LER (HGET/HGETALL)

```
r.hget("usuario:1", "nome")  
r.hgetall("usuario:1")
```

Equivalente a: HGET + HGETALL — lê um campo ou o hash completo

Atividade prática: mini-projeto no Google Colab



Contador de visitas com expiração — todo o CRUD escrito em Python no seu notebook

1 Zerar o contador

```
r.set("visitas:home", 0)
```

2 Simular 5 acessos

```
for _ in range(5):  
    r.incr("visitas:home")  
    print(r.get("visitas:home"))
```

3 Definir expiração de 5 min

```
r.expire("visitas:home", 300)  
print(r.ttl("visitas:home"))
```

4 Cadastrar usuário (hash)

```
r.hset("usuario:1", mapping={"nome": "Seu Nome",  
                             "email": "seu@email.com"})  
print(r.hget("usuario:1", "nome"))
```

Como entregar a atividade



Documento Markdown

Preencha o `template_entrega_aula03.md` com comando/código, saída obtida e print de cada passo.



Repositório da disciplina

Suba o arquivo (e os prints) na pasta pessoal do repositório GitHub da turma.



Reflexão final

Responda em 2–3 linhas por que INCR e EXPIRE fazem sentido para um contador de visitas.

Ferramenta utilizada: RedisInsight, redis-cli ou Google Colab são todos aceitos — indique qual você usou no topo do template.



Próxima aula

05/09/2026 — Chave-valor aplicado: cache, gerenciamento de sessões e mini-projeto com Redis Cloud.

RESUMO DO QUE VIMOS HOJE

- Definição, arquitetura interna e anatomia de chave (key naming)
- Comandos fundamentais: SET, GET, DEL, EXPIRE, INCR, HSET/HGET
- Redis Core x Redis Stack, e as 5 estruturas de dados
- Prática CRUD completa no Redis Cloud via Google Colab (Python)

Dúvidas?